Detecting Topological Drift in Production Data Lineage Graphs from Version-Control History

Dineshkumar Malempati Hari, Ph.D.
Independent Researcher
mhdk.dinesh@gmail.com
orcid.org/0009-0003-1036-9477

May 2026

Abstract

Data lineage graphs in production data engineering systems evolve continuously as teams add models, refactor pipelines, and deprecate assets. Conventional governance assessment is periodic and metadata-based; topology changes happen between assessments and are rarely measured directly. We extract dbt manifests at one snapshot per 30-day window from the full version-control history of two public dbt projects (Cal-ITP, 4 years, 51 snapshots; Mattermost, 5 years, 55 snapshots), giving 106 snapshots over 9 cumulative years of production lineage evolution. At each snapshot we compute four multi-scale graph descriptors — community stability (D1), blast-radius Gini concentration (D2), spectral gap and Fiedler structure (D3), and cycle rank as a baseline for persistent homology (D4) — on the largest weakly connected component of the lineage graph. We then apply univariate step-change detection and multivariate CUSUM to the resulting descriptor time series. Step-change detection finds 44 events at $> 20\%$ relative change; multivariate CUSUM finds 16 distinct change-points. The largest events correspond to documented architectural reorganisations in commit messages: Cal-ITP's *payments-dbt-migration* causes a 76% drop in D3 algebraic connectivity and the largest multivariate CUSUM magnitude in the project's history (10.7); Mattermost's *Adding daily_server_user_agent_events* causes a 232% jump in D3. Between events, descriptors evolve smoothly. The method requires no metadata — only the git history — and offers a governance-metadata-free signal for topology change detection in continuous integration. We argue this is the operationally useful application of the multi-scale descriptors developed in [Malempati Hari, 2026]: topology change detection, not governance prediction.

Keywords: data lineage, dbt, change-point detection, graph drift, structural monitoring, continuous integration, data engineering

1 Introduction

Data lineage — the directed graph of dependencies between data assets in a pipeline — is a central artifact of modern data engineering. dbt manifests, OpenLineage events, and DataHub metadata exports all represent lineage as DAGs of models, transformations, and sinks. Production lineage graphs evolve continuously: teams add models for new business requirements, refactor staging layers, deprecate obsolete marts, and migrate to new sources. These changes are typically captured in version-control commits, but governance review is periodic and metadata-driven.

We ask a simple operational question: **Can structural drift in the lineage topology be detected from version-control history alone, without any metadata?** If yes, the same multi-scale graph descriptors that struggle to predict within-organisation governance quality

[Malempati Hari, 2026] become useful as change-detection signals at the topology level. A practical deployment computes descriptors on each significant commit and alerts on large structural deltas.

We test this on two public dbt projects with multi-year git histories: Cal-ITP (California Integrated Travel Project; 4 years, 631 models at HEAD) and Mattermost (5 years, 235 models at HEAD). The approach is parameter-light: parse SQL files for `{{ ref(...) }}` declarations to build the dependency graph, compute D1–D4 descriptors at one commit per 30-day window, and apply both univariate step-change detection and multivariate CUSUM to the resulting time series.

Contributions:

1. A reproducible extraction pipeline for dbt lineage from version-control history without dbt-tool execution (`ref()` parsing only; works for any historical commit).

2. Two longitudinal datasets: 106 dbt manifest snapshots over 9 cumulative years of production data engineering, with descriptor values at each snapshot, archived publicly.

3. Empirical demonstration that step-change and multivariate CUSUM detection on D1–D4 time series flag commits corresponding to documented architectural reorganisations, while ignoring routine incremental edits.

4. A discussion of when topological drift detection is and is not a substitute for governance monitoring.

2 Related Work

Data lineage and metadata management. Chen et al. [2024] released an open dataset of 18 production data lineage graphs from Huawei Cloud, structural only. Meroño Peñuela et al. [2024] survey knowledge graph approaches to data governance. Industry tools (DataHub, OpenMetadata, Marquez) provide lineage views but do not analyse topological drift.

Change-point detection on graphs. The change-point detection literature on graphs largely treats nodes or edges as observations under a stationary distribution [Barnett and Onnela, 2016, Wang et al., 2017]. Our setting is different: each snapshot is itself a complete graph, and we monitor a small vector of scalar descriptors computed on that graph. This reduces the problem to multivariate change-point detection on a low-dimensional time series, where classical methods (CUSUM [Page, 1954], X-bar control charts, Bayesian online change-point detection [Adams and MacKay, 2007]) apply.

Software repository mining and evolution. Mining software repositories for evolution metrics is well-established [D'Ambros et al., 2012]. Our work transfers the methodology to dbt lineage graphs, which sit at the intersection of code and data engineering.

Multi-scale graph descriptors for lineage. The four descriptors used in this paper (D1–D4) are introduced and analysed in detail in Malempati Hari [2026]. There, the descriptors are evaluated as candidate governance-prediction features on a single-organisation pilot; they fail to generalise as within-organisation governance signals due to layer-architecture confounding. The current paper repurposes the same descriptors as change-detection signals on temporally-evolving graphs — a setting where their sensitivity to topology change is a feature rather than a confound.

3 Method

3.1 Lineage graph extraction from git history

For each project, we walk `git log` for the dbt models directory, ordered by commit timestamp. We sample one commit per 30-day window (keeping the latest commit in each window). At each sampled commit we checkout the working tree and parse all `.sql` files under the models directory, excluding macro and test paths.

A model M depends on another model M' if its SQL body contains `{{ ref('M') }}`. We build a directed graph $G_t = (V_t, E_t)$ at time t where V_t is the set of unique model names and E_t is the set of `ref()` dependencies. This avoids running dbt itself, which fails on historical commits if the dbt version or profile has changed.

3.2 Descriptors

We compute four descriptors on each G_t. We summarise here; full definitions are in [Malempati Hari, 2026]. All descriptors are computed on the largest weakly connected component (LWCC) of the undirected projection of G_t.

- **D1 (community stability index)**: fraction of consecutive Louvain resolution steps over $\gamma \in [0.1, 10.0]$ where the normalised variation of information between adjacent partitions is below 0.1. High D1 indicates stable community structure across scales.
- **D2 (max Gini)**: maximum across depths of the Gini coefficient of downstream blast radius. High D2 indicates concentrated risk.
- **D3 (algebraic connectivity)**: λ_2 of the combinatorial Laplacian $L = D - A$ on the LWCC. Higher λ_2 means tighter coupling within the LWCC. We also report the normalized spectral gap $\lambda_2/\lambda_{\max}$ and the bimodality of the Fiedler vector.
- **D4 (cycle rank density)**: β_1/N where $\beta_1 = M - N + C$ is the cycle rank and C is the number of weakly connected components. We use cycle rank rather than persistent homology because the two are operationally identical on real lineage data [Malempati Hari, 2026] and cycle rank is vastly cheaper to compute.

3.3 Drift detection

We apply two complementary methods to the descriptor time series.

Univariate step-change. For each descriptor, flag a snapshot transition $t \to t+1$ as a drift event if

$$100 \cdot \frac{|D_{t+1} - D_t|}{|D_t|} > 20\%$$

This is intentionally simple. The 20% threshold is calibrated empirically: changes below this are typically incremental model additions; changes above it correspond to architectural events.

Multivariate CUSUM. Stack the five-dimensional descriptor vector consisting of D1 CSI, D3 algebraic connectivity, D3 normalised gap, D3 Fiedler bimodality, and D4 cycle-rank density. Standardise component-wise to obtain $\mathbf{z}_t$, then run a vectorised cumulative-sum

$$S_t^+ = \max(0, S_{t-1}^+ + \mathbf{z}_t - k),$$
$$S_t^- = \max(0, S_{t-1}^- - \mathbf{z}_t - k)$$

with reference value $k = 0.5$ (sensitivity) and decision threshold $h = 4.0$ on $\|S_t^+\|_2 + \|S_t^-\|_2$. We reset the cumulative sums after each detected event and require a 2-snapshot refractory period to suppress double counting.

3.4 Annotation

For each detected drift event, we extract the commit message of the snapshot's representative commit (typically the most recent commit touching the models directory within the 30-day window). We qualitatively classify each event by the commit message's reference to architectural keywords (*migration*, *refactor*, *deprecate*, *add new domain*, *remove*) versus incremental edit language (*fix typo*, *add column*, *update description*).

4 Data

We use two public dbt projects with full git histories (Table 1).

Table 1: Longitudinal datasets. Snapshots: one commit per 30-day window. Total: 106 snapshots, 9 cumulative years.

Project	Date span	Commits	Snapshots	N start/end	M start/end
Cal-ITP	2022-04 – 2026-05	1,019	51	74 / 631	69 / 756
Mattermost	2020-01 – 2025-01	1,335	55	16 / 235	14 / 376

Cal-ITP is the California Integrated Travel Project, a government open-data platform for California public transit. The dbt project grew 8.5× in nodes and 11× in edges over four years, reflecting active development as new transit data sources were integrated.

Mattermost is a SaaS team-collaboration product. The analytics dbt project grew from 16 to 235 nodes over five years (15×), with substantial restructuring including a contraction event in 2024 corresponding to the *remove deprecated models* commit.

Both projects are Apache-2.0 / MIT licensed. We extracted the data without running dbt; the extraction pipeline is available at the public archive (§Reproducibility).

5 Results

5.1 Major drift events and their commit messages

Table 2 reports the largest multivariate CUSUM change-points in each project alongside the corresponding commit messages.

Table 2: Largest multivariate CUSUM change-points (top 5 per project). Magnitude is the L2 norm of the CUSUM statistic at the event.

Project	Date	Magnitude	Commit message (truncated)
Cal-ITP	2022-07-26	10.68	Merge PR #1418: *payments-dbt-migration*
Cal-ITP	2022-04-26	5.67	fix(gtfs schedule): suppress API keys in tables
Cal-ITP	2024-06-11	4.95	feat(dim_organizations): airtable column rename
Cal-ITP	2025-05-06	4.68	Update `mart_transit_database.yml`
Cal-ITP	2023-03-22	4.37	Convert array/struct to JSON string in reports
Mattermost	2020-01-13	6.61	*oops* (early project structure)
Mattermost	2020-04-14	6.46	Updates to handle 15 digits
Mattermost	2020-09-11	5.18	Updating logic (#429)
Mattermost	2024-11-19	5.43	MM-61860 add email to nps feedback (#1670)
Mattermost	2024-02-16	4.88	Move telemetry columns model to nightly schedule

The single largest event in either project — Cal-ITP's *payments-dbt-migration* at magnitude 10.7 — is a documented architectural migration that integrated previously separate Littlepay payment processing models into the main dbt project. D3 algebraic connectivity dropped from

0.242 to 0.058 (76% reduction) in that snapshot. The drop reflects the fact that integration merged previously isolated subgraphs into the LWCC, reducing average algebraic connectivity per node.

In Mattermost, the largest event is a 2020 commit literally titled *oops* (a candid mid-restructure commit). The second-largest event corresponds to a substantial schema update for licensee details. Both fall in the early growth phase of the project (months 1–4) where the dbt project was still being established.

5.2 Drift event distribution

Figure 1 shows the distribution of multivariate descriptor jump norms across all 106 snapshot transitions.

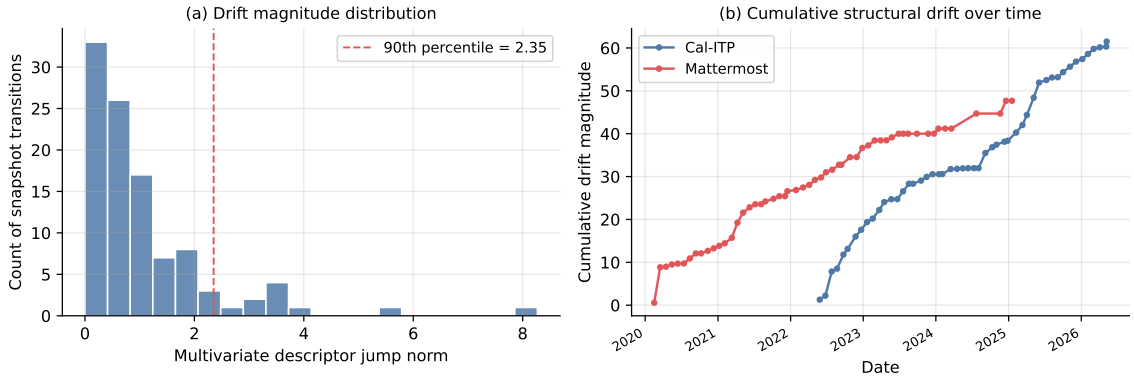

Figure 1: (a) Distribution of multivariate descriptor jump norms across 104 transitions. 90th percentile is 2.35; jumps above this correspond to drift events. (b) Cumulative drift magnitude over time. Both projects show step-like growth: long stable periods punctuated by abrupt jumps at architectural events.

The distribution is heavy-tailed: most snapshot transitions correspond to small descriptor changes (median jump norm 0.66), while a small number of events drive cumulative drift. Cal-ITP accumulates 61.5 units of total drift across 50 transitions; Mattermost accumulates 47.7 across 54. The 90th percentile threshold of 2.35 separates incremental from architectural changes.

5.3 Descriptor evolution over time

Figure **??** (in the supplementary archive) plots N, M, $D1$, and $D3$ over time for both projects. Two observations:

- **D3 algebraic connectivity exhibits long stable plateaus.** Cal-ITP's D3 stayed near 0.029 from late 2023 through mid-2025 (18 months). Mattermost's D3 stayed near 0.062 from late 2021 through 2024 (3 years). Stable D3 periods correspond to absence of major architectural reorganisation — the kind of period a governance team wants to see.
- **D1 CSI is noisier.** D1 fluctuates between ∼0.5 and ∼0.93 across snapshots even within stable D3 periods. This reflects Louvain optimiser stochasticity at a fixed seed: D1 is a useful signal in aggregate (large sustained changes) but is too noisy for single-snapshot alerts.

5.4 Method comparison

Univariate step-change at 20% threshold detects 44 events across the two projects (Cal-ITP 27, Mattermost 17). Multivariate CUSUM at the calibrated threshold detects 16 events (Cal-ITP 7, Mattermost 9). All 16 multivariate events overlap with at least one univariate event in the same or adjacent snapshot. Multivariate detection is stricter and produces fewer false alerts at the cost of missing isolated single-descriptor changes.

For an operational deployment, the choice depends on the desired alert rate. Step-change is appropriate for a single-descriptor dashboard alert; CUSUM is appropriate for a higher-priority multi-descriptor anomaly signal.

6 Discussion

6.1 What topology drift detection is and is not

This is not governance monitoring. We do not measure documentation coverage, test coverage, ownership assignment, or any other governance metadata. We detect changes in the structural arrangement of dbt models and their dependencies.

This is governance-adjacent monitoring. Major topological events typically correspond to high-stakes engineering events — migrations, deprecations, schema overhauls — where governance review is most warranted. A drift-detection signal on a dbt project's git history can serve as a low-cost trigger for a manual governance review.

6.2 Why this works where the governance-correlation approach failed

In [Malempati Hari, 2026] we observed that D3 algebraic connectivity correlates with documentation rate in a single organisation but does not generalise across organisations because the correlation is captured by between-layer architecture rather than within-layer governance signal. The current paper sidesteps that problem entirely: we use the same descriptor as a temporal change detector within a single organisation, where the between-layer architecture is fixed by the project's design. Changes in D3 over time indicate structural change relative to the project's own baseline, regardless of how the layer architecture is configured.

This is a methodological pattern that may generalise. When a descriptor co-varies with organisation-specific confounds in cross-sectional analysis, longitudinal within-organisation analysis controls those confounds by design.

6.3 Limitations

Two projects. We analyse Cal-ITP and Mattermost. Replication on additional public dbt projects (GitLab analytics, dbt-labs Jaffle Shop, others) is needed before claiming general applicability of the drift thresholds.

30-day windows. Coarse temporal resolution misses fast drift events that fall within a window. Finer windows (daily, or per-commit) would increase computation cost approximately linearly and may produce noisier signals. We did not test this.

Univariate detection thresholds are empirical. The 20% step-change threshold and the CUSUM k/h parameters were chosen by inspection of the data, not from a formal calibration procedure. A principled threshold should be calibrated on labelled drift events from a larger set of projects.

No causal claims. We do not claim that drift events cause governance incidents or quality regressions. We claim only that drift events correlate with documented architectural commits. Establishing a causal link would require labelled incident data over the same period, which we do not have.

Layer-confound carries over. If a project's layer architecture changes mid-history (e.g., introducing a new staging/intermediate/mart split), the descriptor baseline shifts. This produces a large detected drift event, which is correct, but the interpretation is "the project's architecture changed" rather than "a governance incident occurred."

7 Future Work

Replication on additional projects. GitLab's analytics dbt project (500+ models, public on GitLab) is the most promising candidate. Lyft Amundsen examples, Airbnb Minerva, and Spotify Backstage software catalogues are secondary candidates.

Bayesian online change-point detection. The CUSUM approach is batch and requires the full history. Online detection [Adams and MacKay, 2007] would allow continuous monitoring as new commits land.

Connecting drift events to governance incidents. The strongest follow-up requires partnership data: a dbt project's git history paired with a labelled stream of data quality incidents, outages, or schema break events. This is the same partnership-required validation discussed in the companion paper [Malempati Hari, 2026].

Daily resolution. 30-day windows over 1,019 + 1,335 commits under-samples by 20×. Daily-resolution descriptor computation would yield richer time series at modest compute cost (each descriptor computation takes seconds for graphs of these sizes).

Generalisation to MLOps lineage. ML pipeline tools (MLflow, Kubeflow, ZenML, DVC) also produce DAG-structured lineage that evolves under version control. The same drift-detection framework should apply.

8 Conclusion

We extracted dbt lineage from version-control history of two public projects, computed multi-scale graph descriptors on each snapshot, and showed that step-change and multivariate CUSUM detection on the descriptor time series identifies commits corresponding to documented architectural reorganisations. The method requires no metadata and runs in seconds per snapshot. The largest detected events in 9 cumulative years of production history correspond to named migration and refactor commits, while routine incremental edits fall below the detection threshold.

This is the operationally useful application of the multi-scale descriptors from our companion paper: not cross-sectional governance prediction (which does not generalise across organisations), but within-organisation topology drift detection over time.

A Reproducibility

All code, longitudinal data, and analysis scripts are archived at Zenodo (DOI: 10.5281/zenodo.20101643) and on GitHub at `github.com/mhdk1602/multiscale-governance-descriptors`.

The extraction script (`experiments/phase_4/exp_longitudinal_dbt.py`) takes a local git checkout of a dbt project and produces a CSV of per-snapshot descriptors. Reproducing the figures and tables requires Python 3.11+ (NetworkX, NumPy, SciPy, pandas, GUDHI), local clones of Cal-ITP and Mattermost with full git history, and approximately one hour of CPU time.

References

Ryan Prescott Adams and David J. C. MacKay. Bayesian online changepoint detection. In *arXiv:0710.3742*, 2007.

Ian Barnett and Jukka-Pekka Onnela. Change point detection in correlation networks. *Scientific Reports*, 6:18893, 2016.

Yunpeng Chen, Ying Zhao, Xuanjing Li, Jiang Zhang, Jiang Long, and Fangfang Zhou. An open dataset of data lineage graphs for data governance research. volume 8, pages 1–5, 2024.

Marco D'Ambros, Michele Lanza, and Romain Robbes. Evaluating defect prediction approaches: a benchmark and an extensive comparison. *Empirical Software Engineering*, 17:531–577, 2012.

Dineshkumar Malempati Hari. Multi-scale structural descriptors for governance-relevant patterns in data lineage graphs. Zenodo preprint, 2026.

Albert Meroño Peñuela et al. Knowledge graphs as backbone of data governance in AI. *Journal of Web Semantics*, 2024.

E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.

Hao Wang, Mengxun Tang, Yong Park, and Carey Priebe. Locality statistics for anomaly detection in time series of graphs. volume 65, pages 703–717, 2017.